

MỤC LỤC

MỤC LỤC.....	1
Giai đoạn 4: JavaScript - Chuẩn bị cho ReactJS.....	2
1. Biến, vòng lặp, rẽ nhánh, hàm, mảng, chuỗi.....	2
a. Biến (Variables).....	2
b. Vòng lặp (Loops).....	2
c. Rẽ nhánh (Conditionals).....	3
d. Hàm (Functions).....	3
e. Mảng (Arrays).....	4
2. Callback và callback hell.....	5
3. OOP: Class, object, reference.....	6
a. Class.....	6
b. Object.....	6
c. Reference.....	7
4. DOM.....	7
a. Khái niệm.....	7
b. Truy cập phần tử DOM.....	7
c. Thay đổi nội dung và thuộc tính.....	7
d. Thêm hoặc xóa phần tử DOM.....	8
e. Lắng nghe sự kiện (Event Listener).....	8
5. JSON (JavaScript Object Notation).....	9
6. Promise, Promise.all, async, await.....	9
a. Promise.....	9
b. Promise.all().....	10
c. async / await.....	10
7. Fetch API.....	11
a. Khái niệm.....	11

Giai đoạn 4: JavaScript - Chuẩn bị cho ReactJS

1. Biến, vòng lặp, rẽ nhánh, hàm, mảng, chuỗi

a. Biến (Variables)

Biến dùng để lưu trữ dữ liệu. Có 3 từ khóa khai báo:

- **var**: phạm vi toàn hàm (function scope), có thể bị hoisting.
- **let**: phạm vi khối (block scope), dùng trong ES6 trở đi.
- **const**: giống let nhưng không thể gán lại giá trị.



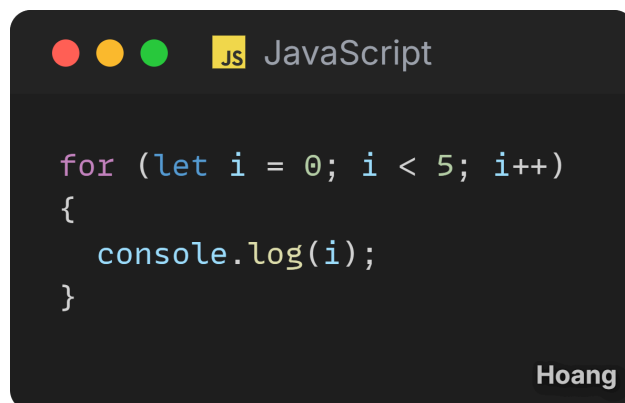
A screenshot of a code editor window titled "JavaScript" with a yellow "JS" icon. The code inside shows two variable declarations: `let name = "Meo";` and `const age = 20;`. The editor has a dark background and the name "Hoang" is visible in the bottom right corner.

b. Vòng lặp (Loops)

Dùng để lặp đi lặp lại một khối lệnh.

Phổ biến nhất: for, while, do...while, for...of, for...in.

Ví dụ:



A screenshot of a code editor window titled "JavaScript" with a yellow "JS" icon. The code inside shows a for loop: `for (let i = 0; i < 5; i++) { console.log(i); }`. The editor has a dark background and the name "Hoang" is visible in the bottom right corner.

c. Rẽ nhánh (Conditionals)

Giúp chương trình quyết định luồng chạy, chủ yếu dùng if, else.

Ví dụ:



```

if (age > 18)
{
    console.log("Người lớn");
}
else
{
    console.log("Trẻ con");
}

```

Hoang

d. Hàm (Functions)

Đóng gói logic để tái sử dụng. Có 3 cách viết phổ biến:

- Function declaration
- Function expression
- Arrow function (ES6)

Ví dụ:



```

// Function Declaration
function sayHello() {
    console.log("Hello từ function declaration!");
}

// Function Expression
const sayHi = function() {
    console.log("Hello từ function expression!");
};

// Arrow Function (ES6)
const greet = () => {
    console.log("Hello từ arrow function!");
};

// Gọi 3 hàm
sayHello();
sayHi();
greet();

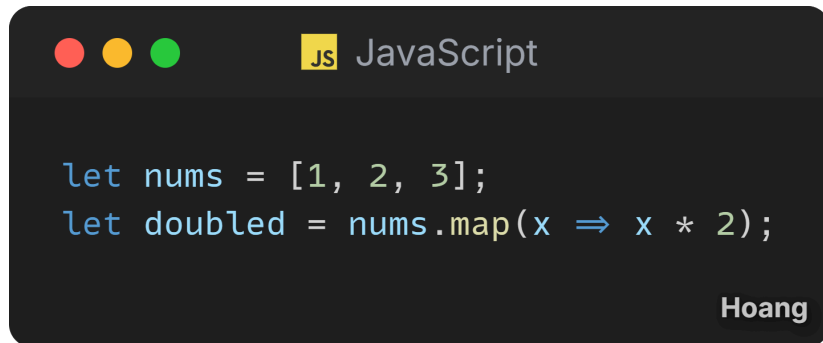
```

Hoang

e. Mảng (Arrays)

Lưu danh sách các phần tử, thường dùng các hàm như `map()`, `filter()`, `reduce()`, `forEach()`.

Ví dụ:



```
let nums = [1, 2, 3];
let doubled = nums.map(x => x * 2);
```

Hoang

f. Xâu (Strings)

Xử lý chuỗi với `length`, `slice`, `split`, `replace`, `includes`, `template literals`:



```
let text = "Xin chào JavaScript!";

// Độ dài chuỗi
console.log(text.length); // 19

// Cắt chuỗi
console.log(text.slice(4, 9)); // "chào"

// Tách chuỗi thành mảng
console.log(text.split(" ")); // ["Xin", "chào", "JavaScript!"]

// Thay thế nội dung
console.log(text.replace("JavaScript", "ReactJS")); // "Xin chào ReactJS!"

// Template literals
let name = "Mèo";
console.log(`Hello ${name}, học JS vui nhé!`);
```

Hoang

2. Callback và callback hell

Callback là một hàm được truyền làm đối số cho hàm khác, và được gọi lại (callback) sau khi hàm kia hoàn thành. Nó giúp xử lý các tác vụ bất đồng bộ (asynchronous), như đọc file, gọi API, setTimeout...

Ví dụ: done() là callback, được gọi sau khi in lời chào.



```

function greet(name, callback) {
  console.log("Xin chào " + name);
  callback();
}

function done() {
  console.log("Hoàn tất lời chào!");
}

greet("Mèo", done);
  
```

Hoang

Callback Hell là khi ta lồng nhiều callback bên trong nhau, khiến code rối, khó đọc, khó bảo trì. Thường gặp khi thực hiện nhiều thao tác bất đồng bộ liên tiếp.

Người ta thường dùng **Promise** hoặc **async/await** để tránh callback hell.



```

getUser(id, (user) => {
  getPosts(user, (posts) => {
    getComments(posts, (comments) => {
      console.log("Xong hết!");
    });
  });
});
  
```

Hoang

3. OOP: Class, object, reference

a. Class

Class là khuôn mẫu dùng để tạo đối tượng (object). Nó giúp code rõ ràng, dễ tái sử dụng và mô phỏng hướng đối tượng.

```
JavaScript

// Class - định nghĩa khuôn mẫu đối tượng
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  // Method trong class
  greet() {
    console.log(`Xin chào, mình là ${this.name}, ${this.age} tuổi.`);
  }
}
```

Hoang

b. Object

Object là thể hiện (instance) của class. Mỗi object có dữ liệu riêng, nhưng dùng chung cấu trúc từ class.

```
JavaScript

const user1 = new User("Mèo", 20);
user1.greet(); // 👉 "Xin chào, mình là Mèo, 20 tuổi."
```

Hoang

c. Reference

Trong JavaScript, object được lưu theo tham chiếu (reference). Khi gán một object cho biến khác, cả hai cùng trỏ đến cùng vùng nhớ, nên thay đổi một biến sẽ ảnh hưởng đến biến kia.



```

const user2 = user1;
user2.name = "Tôm";

console.log(user1.name); // 📌 "Tôm"

```

Hoang

4. DOM

a. Khái niệm

DOM là mô hình cho phép JavaScript tương tác và thao tác với nội dung HTML của trang web. Khi trình duyệt tải trang, nó biến mã HTML thành một cây đối tượng (object tree), nơi mỗi thẻ HTML là một node (nút). JS có thể truy cập, chỉnh sửa, thêm, xóa các phần tử này trực tiếp.

b. Truy cập phần tử DOM

Các phương thức thường dùng:



```

document.getElementById("title");
document.querySelector(".btn");
document.querySelectorAll("p");

```

Hoang

c. Thay đổi nội dung và thuộc tính

Có thể cập nhật văn bản, style hoặc thuộc tính HTML ngay từ JS:



```

const title = document.getElementById("title");
title.textContent = "Xin chào JavaScript!";
title.style.color = "blue";

```

Hoang

d. Thêm hoặc xóa phần tử DOM

```
const newItem = document.createElement("li");
newItem.textContent = "Item mới";
document.querySelector("ul").appendChild(newItem);
```

Hoang

e. Lắng nghe sự kiện (Event Listener)

DOM cho phép bắt các hành động của người dùng như click, nhập liệu, cuộn trang...



```
const btn = document.querySelector(".btn");
btn.addEventListener("click", () => {
  alert("Bạn vừa nhấn nút!");
});
```

Hoang

5. JSON (JavaScript Object Notation)

Khái niệm: JSON là định dạng dữ liệu dạng văn bản, dùng để trao đổi dữ liệu giữa client và server. Nó rất phổ biến trong frontend vì JavaScript đọc và xử lý JSON rất dễ.

Cấu trúc JSON gần giống object trong JS, nhưng key phải đặt trong dấu ngoặc kép và chỉ hỗ trợ các kiểu dữ liệu cơ bản như string, number, boolean, null, array, object.



```
{
  "name": "Mèo",
  "age": 20,
  "isStudent": true,
  "address": {
    "city": "Hà Nội",
    "district": "Cầu Giấy"
  },
  "skills": ["HTML", "CSS", "JavaScript"],
  "contact": {
    "email": "meo@example.com",
    "github": "https://github.com/meo"
  }
}
```

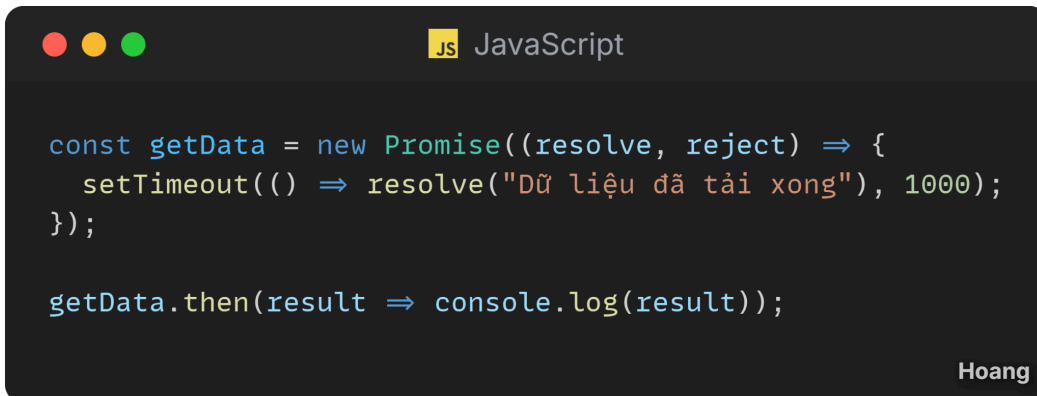
6. Promise, Promise.all, async, await

a. Promise

Promise là một cách xử lý bất đồng bộ (asynchronous) trong JavaScript, giúp tránh callback hell.

Một Promise có 3 trạng thái:

- *pending* (đang chờ)
- *fulfilled* (thành công)
- *rejected* (thất bại)



```
const getData = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Dữ liệu đã tải xong"), 1000);
});

getData.then(result => console.log(result));
```

b. Promise.all()

Dùng để **chạy nhiều Promise song song** và chỉ trả kết quả khi tất cả đều hoàn thành.
Nếu một Promise lỗi, cả nhóm sẽ bị reject.

A screenshot of a code editor window titled "JavaScript" with a yellow "JS" icon. The code inside is:

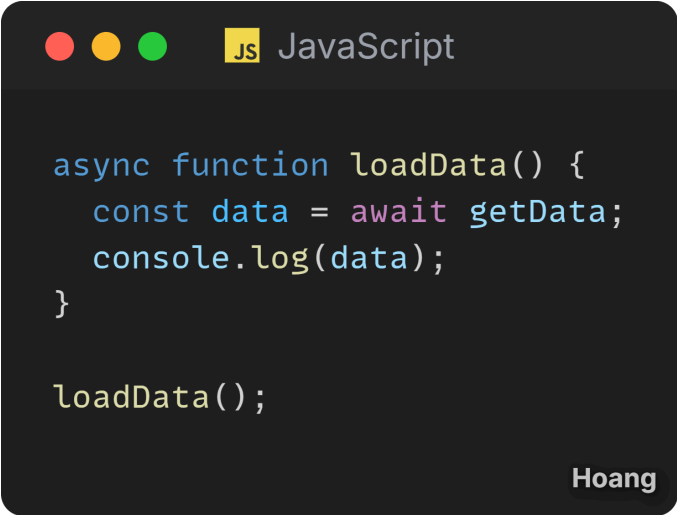
```
const p1 = Promise.resolve("User");
const p2 = Promise.resolve("Posts");

Promise.all([p1, p2]).then(results => console.log(results));
// 👉 ["User", "Posts"]
```

The editor has a dark background and the name "Hoang" is visible in the bottom right corner.**c. async / await**

Cú pháp hiện đại hơn giúp viết code bất đồng bộ trông giống code đồng bộ.

- async khai báo hàm trả về Promise.
- await dừng hàm lại cho đến khi Promise hoàn thành.

A screenshot of a code editor window titled "JavaScript" with a yellow "JS" icon. The code inside is:

```
async function loadData() {
  const data = await getData;
  console.log(data);
}

loadData();
```

The editor has a dark background and the name "Hoang" is visible in the bottom right corner.

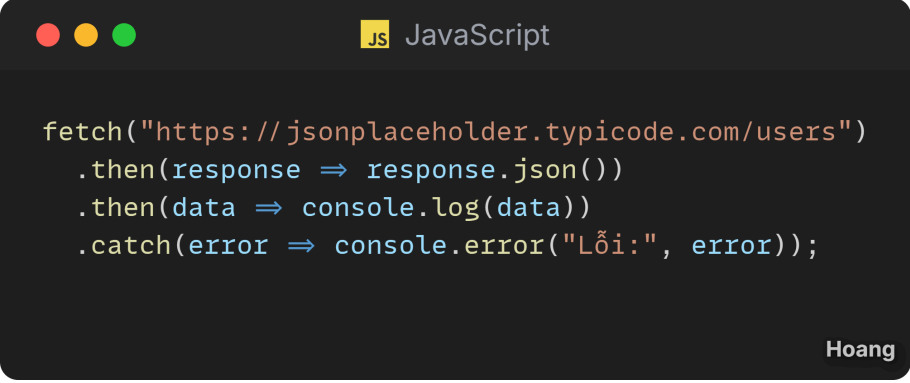
7. Fetch API

a. Khái niệm

`fetch()` là hàm tích hợp trong JavaScript giúp **gửi yêu cầu HTTP đến server để lấy hoặc gửi dữ liệu** (thường là JSON).

Nó trả về một Promise, nên có thể dùng với `.then()` hoặc `async/await`.

Ví dụ cơ bản với `.then()`



```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error("Lỗi:", error));
```

Ví dụ với `async/await` (code gọn, dễ đọc hơn)



```
async function getUsers() {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error("Lỗi:", err);
  }
}

getUsers();
```

Gửi dữ liệu (POST request)



```
async function createUser() {
  const res = await fetch("https://jsonplaceholder.typicode.com/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: "Mèo", age: 20 })
  });
  const data = await res.json();
  console.log(data);
}
```

Code JS cho giao diện của tuần trước.

https://youtu.be/f9S3Xq8_5Wg